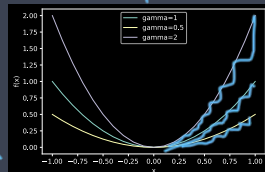


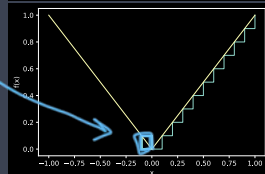
» Summary

- * No “one size fits all” choice of step size.
 - * Scaling function e.g. from x^2 to $2x^2$ changes magnitude of derivative and so of steps taken
- * Maybe need to change the step size over time
 - * If derivative doesn't decrease as get close to minimum then can end up “bouncing around”
 - * If derivative decreases too quickly as get close to minimum then can end up with v slow convergence
- * Convergence rate can be v sensitive to properties of function being minimised
 - * Quadratic function is “best” case, exponentially fast convergence (jargon: quadratic-like $\rightarrow$ strongly convex)
 - * Kinks and flat areas in functions can greatly slow convergence (jargon: has kinks $\rightarrow$ non-smooth)

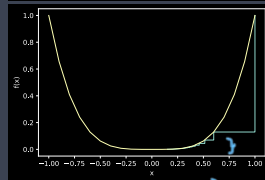
$(x, f(x))$ single variable $\Rightarrow$ curve



$2x^2$
 x^2
 $0.5x^2$



$|x|$



}

» Quadratic With Two Inputs

So far we only looked at functions with scalar input x , let's look at functions where x is a vector with two elements

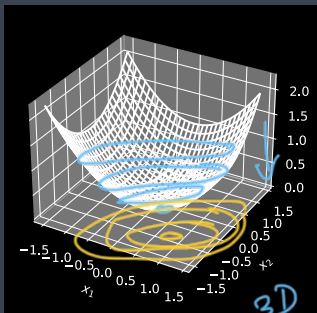
$$* f(x) = x_1^2 + x_2^2, x = [x_1, x_2].$$

Dual variable
⇒ surface

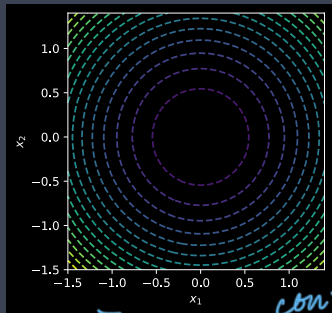


2=1

①



3D



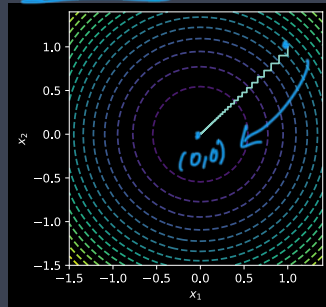
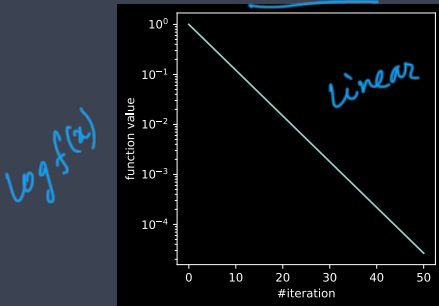
②

contour

- * Its often easier/clearer to use a contour plot instead of a 3D plot
- * Each contour line indicates points at which the function has the same value

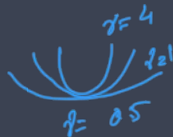
» Quadratic With Two Inputs

- * $f(x) = x_1^2 + x_2^2$, $x = [x_1, x_2]$.
- * Starting value $x_0 = [1, 1]$, step size $\alpha = 0.1$

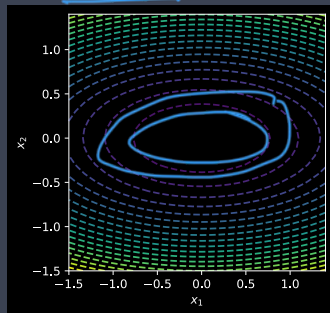
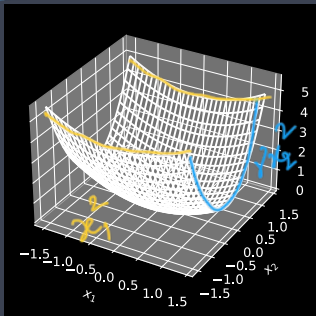


- * $\log f(x)$ decreases linearly i.e. $f(x)$ decreases exponentially.
- * On contour plot x follows a path perpendicular to contour lines – this is path of steepest descent

» Quadratic With Two Inputs



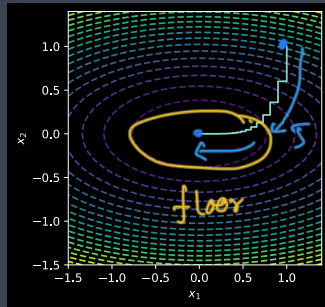
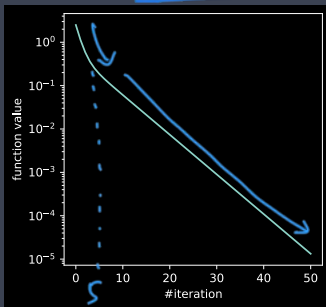
- * Now let's “squash” bowl in one direction: $f(x) = x_1^2 + \gamma x_2^2$, $x = [x_1, x_2]$.
- * $\gamma = 1$ gives previous example, let's try $\gamma = 4$:



$\gamma = 4$

» Quadratic With Two Inputs

- * $f(x) = x_1^2 + \gamma x_2^2$, $x = [x_1, x_2]$. $\gamma = 4$
- * Starting value $x_0 = [1, 1]$, step size $\alpha = 0.1$

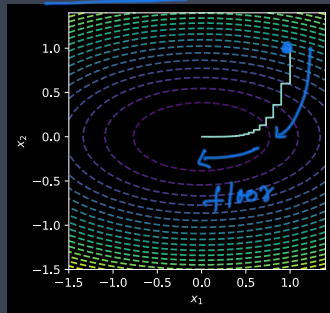
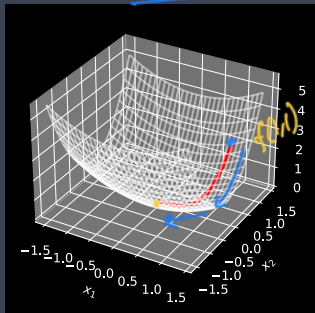


- * Function initially decreases quickly, then more slowly. But decrease of $\log f(x)$ is still linear-like
- * On contour plot x follows a path perpendicular to contour lines – path of steepest descent – but this path is now curved. Why?

» Quadratic With Two Inputs

* $f(x) = x_1^2 + \gamma x_2^2$, $x = [x_1, x_2]$. $\gamma = 4$

* Starting value $x_0 = [1, 1]$, step size $\alpha = 0.1$



» Machine Learning Cost Functions

Linear Regression/Least Squares

→ * Training data: $(x^{(i)}, y^{(i)})$, $i = 1, \dots, m$. $y^{(i)}$ is real-valued.

* Cost function: $J(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2 = f(\theta^2) = f(\theta_1, \theta_2)$

* Cost function is quadratic in θ , so expect nice convergence behaviour

* Note: we've switched to our ML notation, $J(\theta)$ instead of $f(x)$
i.e. our optimisation task is now

rather than $\min_{\theta} J(\theta)$

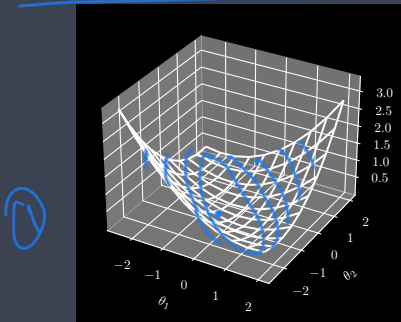
→ $\min_x f(x)$
(just a change in variable names)

⇕
Equivalent
optimization
of
cost fns

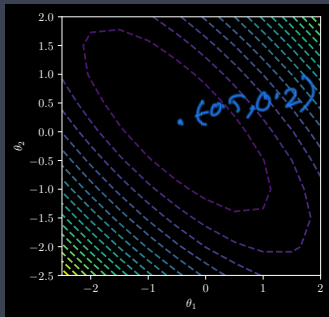
» Machine Learning Cost Functions

Linear Regression/Least Squares

- * Training data: $(\mathbf{x}^{(i)}, y^{(i)})$, $i = 1, \dots, m$. $y^{(i)}$ is real-valued.
- * Cost function: $J(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$ ✓
- * Cost function is quadratic in θ , so expect nice convergence behaviour
- * $y^{(i)} = \theta^T \mathbf{x}^{(i)}$, $\theta = [-0.5, 0.2]$, $m = 1000$ training data points with random $\mathbf{x}^{(i)}$, $i = 1, \dots, n$



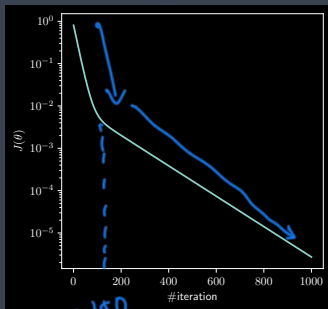
Surface



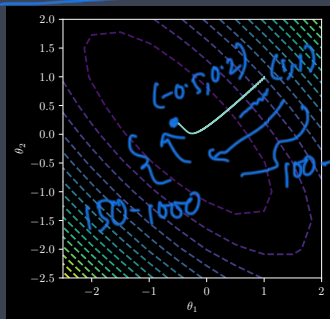
Contour

» Example: Linear Regression/Least Squares

- * $y^{(i)} = \theta^T x^{(i)}$, $\theta = [-0.5, 0.2]$, $m = 1000$ training data points with random $x^{(i)}$, $i = 1, \dots, n$
- * Starting value $x_0 = [1, 1]$, step size $alpha = 0.1$



100-150



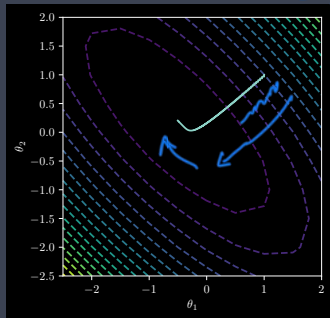
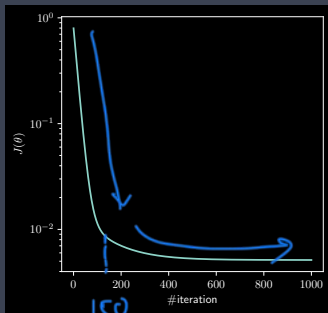
» Example: Linear Regression/Least Squares

Now add some observation noise

- * $y^{(i)} = \theta^T x^{(i)} + n^{(i)}$, $\theta = [-0.5, 0.2]$, noise $n^{(i)}$ is normal with mean 0, std dev 0.1

$\mu=0, \sigma=0 \rightarrow$ no change
 $\mu=0, \sigma=0.1 \rightarrow$ mod. change
 $\mu=0, \sigma=0.5 \rightarrow$ huge change

Gaussian
 $\frac{1}{\sigma\sqrt{2\pi}}$
 $\frac{1}{\sigma}$
 σ
 mean



- * Notice that minimum of $J(\theta)$ is now about $0.1^2 = 0.01 = 10^{-2}$. When estimated parameters are $[-0.5, 0.2]$ then

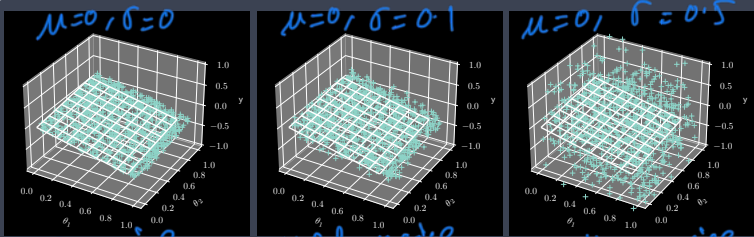
$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2 = \frac{1}{m} \sum_{i=1}^m (n^{(i)})^2$$

- * Stop optimisation loop once get close to this noise floor

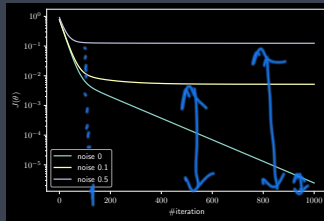
» Example: Linear Regression/Least Squares

- * Impact of varying the amount of measurement noise:

$$y^{(i)} = \theta^T x^{(i)} + n^{(i)}, \text{ noise } n^{(i)} \text{ std dev } 0, 0.1, 0.5:$$



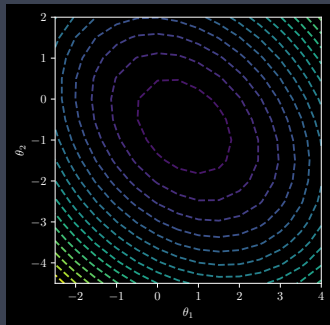
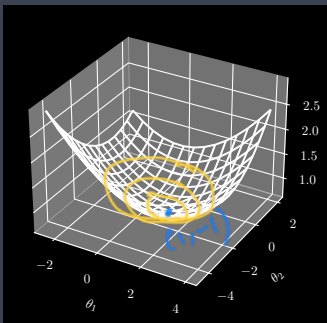
- * And corresponding effect on cost function:



- * Min value of cost function increases as noise increases.

» Machine Learning Cost Functions: Logistic regression

- * Training data: $(x^{(i)}, y^{(i)})$, $i = 1, \dots, n$. $y^{(i)}$ is +1 or -1
- * Cost function: $J(\theta) = \sum_{i=1}^m \log(1 + e^{-y^{(i)} \theta^T x^{(i)}}) + 0.5C \sum_{i=1}^n \theta_i^2$
(using L2 regularisation)
- * $y^{(i)} = \text{sign}(\theta^T x^{(i)})$, $\theta = [1, -1]$, $m = 1000$ training data points with random $x^{(i)}$, $C = 0.1$



L2

quadratic

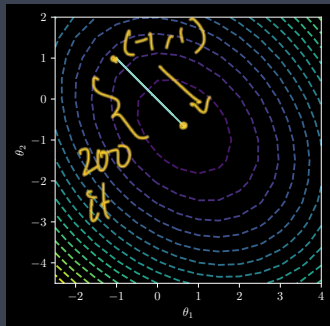
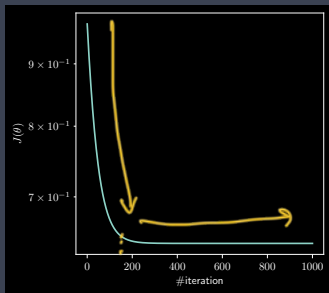
L2

2

①

» Machine Learning Cost Functions: Logistic regression

- * $y^{(i)} = \text{sign}(\theta^T x^{(i)})$, $\theta = [1, -1]$, $m = 1000$ training data points with random $x^{(i)}$, $C = 0.1$
- * Starting value $x_0 = [-1, 1]$, step size $\text{alpha} = 0.1$

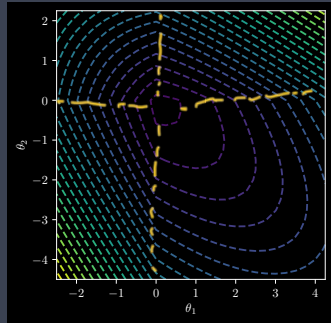
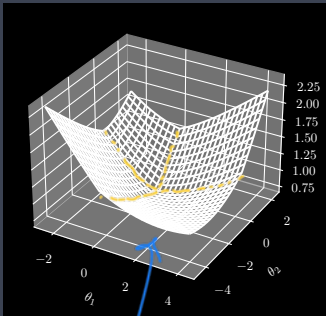


- * Again, stop once get close to the noise floor – after about 200 iterations

» Machine Learning Cost Functions: Logistic regression

What if we change to using L1 regulariser (LASSO)?

- * Cost function: $J(\theta) = \sum_{i=1}^m \log(1 + e^{-y^{(i)}\theta^T x^{(i)}}) + C \sum_{i=1}^n |\theta_i|$
- * Remember, $|\theta_i|$ is *non-smooth*.
- * $y^{(i)} = \text{sign}(\theta^T x^{(i)})$, $\theta = [1, -1]$, $m = 1000$ training data points with random $x^{(i)}$, $C = 0.1$

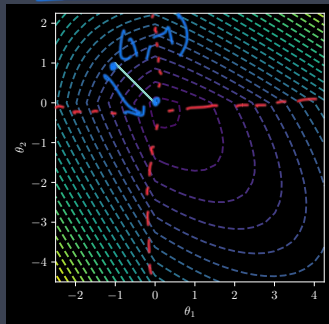
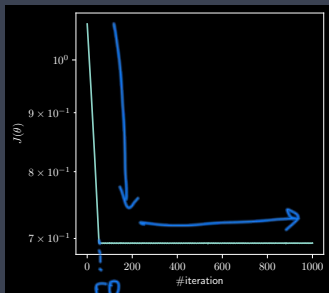


* Notice the kinks ...

» Machine Learning Cost Functions: Logistic regression

* L1 regulariser

- * $y^{(i)} = \text{sign}(\theta^T x^{(i)})$, $\theta = [1, -1]$, $m = 1000$ training data points with random $x^{(i)}$, $C = 0.1$
- * Starting value $x_0 = [-1, 1]$, step size $\alpha = 0.1$



- * Despite kinks/non-smoothness, convergence is fast - Logistic regression+L1 is well-behaved even if kinks can cause problems in general
- * Again, stop once get close to the noise floor - after about 50 iterations

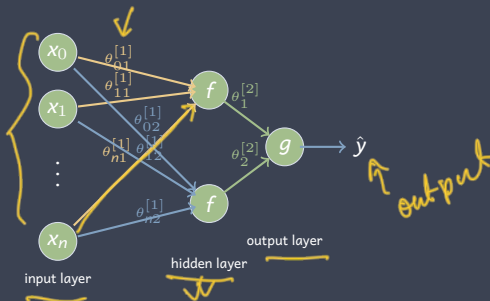
» Summary So Far

- * Linear and logistic regression (with L2 regulariser) are similar in kind to quadratic examples – can expect fast convergence
- * Using L1 regulariser adds kinks, so potentially makes convergence slow/causes “chatter” around minimum, but ok when used with linear and logistic regression.
- * We still have:
 - * No “one size fits all” choice of step size.
 - * Maybe need to change the step size over time



» Machine Learning Cost Functions: Neural Networks

How about neural nets? Recall neural network with one-hidden layer:



$$z_1 = f(\theta_{01}^{[1]}x_0 + \theta_{11}^{[1]}x_1 + \dots + \theta_{n1}^{[1]}x_n)$$

$$z_2 = f(\theta_{02}^{[1]}x_0 + \theta_{12}^{[1]}x_1 + \dots + \theta_{n2}^{[1]}x_n)$$

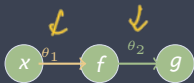
$$\hat{y} = g(\theta_1^{[2]}z_1 + \theta_2^{[2]}z_2)$$

- * f and g are activation and output functions, respectively. Typical choices for f are ReLu and Sigmoid. Typical choice for g is Sigmoid.

» Machine Learning Cost Functions: Neural Networks

Toy NN.

- * Toy example: one input, one hidden node, one output, two parameters/weights:

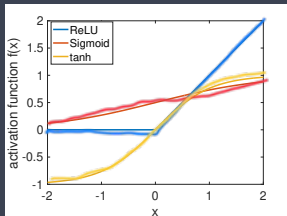


$$z = f(\theta_1 x + \theta_2), \hat{y} = g(z)$$

→ * ReLU (Rectified Linear Unit) $f(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$

→ * Sigmoid $g(x) = \frac{e^x}{1+e^x}$

- * For classification problems predict +1 when $\frac{e^x}{1+e^x} > 0.5$, -1 when $\frac{e^x}{1+e^x} < 0.5$



ReLU
tanh
Sigmoid

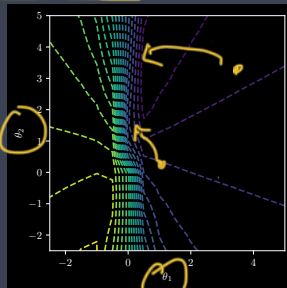
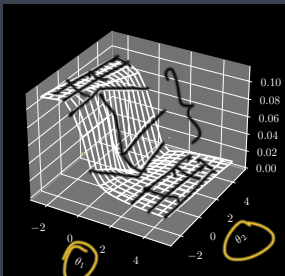
» Machine Learning Cost Functions: Neural Networks

* Toy example:

$$z = f(\theta_1 x + \theta_2), \hat{y} = g(z)$$

✓ * $\theta = [1, 5]$, f is ReLu, g is sigmoid, $m = 1000$ training data points with random $x^{(i)}$

* Quadratic loss function (as with linear regression):

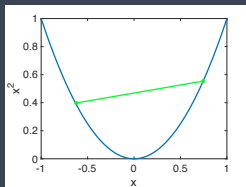


- * Observe the complex nature of the cost function: not a simple “bowl” quadratic shape.
- * Flat areas, narrow valleys, kinks → might be hard to find θ that minimises cost!

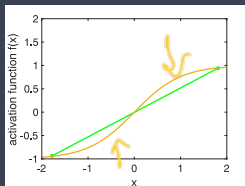
» Convexity vs Non-convexity

The real watershed in optimisation is between minimising convex and non-convex functions.

- * Convex function: when draw a line between any two points on function the line lies *above* the function. E.g.



- * Sigmoid is not convex:



- * Convexity ensures function has a global minimum, and gradient descent will find this (still need to choose step size well, and convergence rate can depend on strong convexity, non-smoothness etc).

» Convexity vs Non-convexity

- * Non-convex optimisation - Here be Dragons!

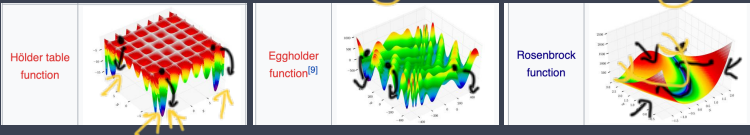


- * See https://en.wikipedia.org/wiki/Test_functions_for_optimization for some nasty examples:

②

③

①

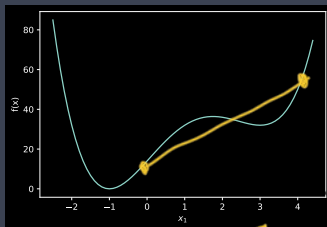


- * Neural nets are non-convex, so expect optimisation to be harder than for linear/logistic regression.
- * But no need to abandon hope - while in general non-convex functions can be v nasty indeed, we're mostly just interested in the special case of neural nets - hopefully the range of possible non-convex functions is more limited (although still large).
- * Plus we know that gradient descent will still tend to converge to a local minimum even when function is non-convex ...

» Example: Convergence To Local Minimum

↙ +VR

$$* (x-1)(x+2)(x-3)^2 + 32$$



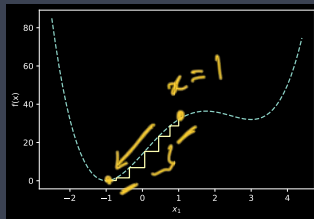
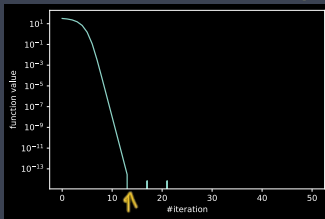
↗

↗

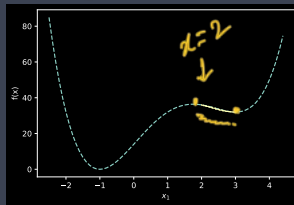
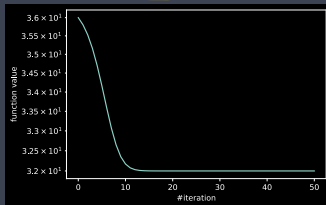
» Example: Convergence To Local Minimum

* $f(x) = (x-1)(x+2)(x-3)^2 + 32$ $\leftarrow +ve$

* Starting value $x_0 = 1$, step size $\alpha = 0.02$



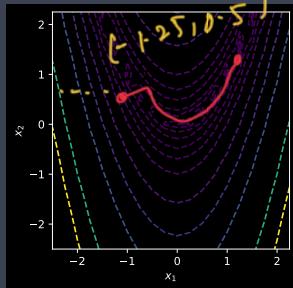
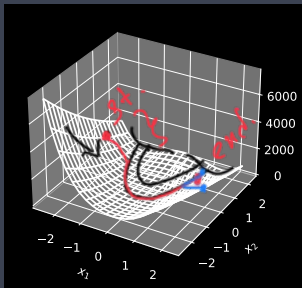
* Starting value $x_0 = 2$, step size $\alpha = 0.02$



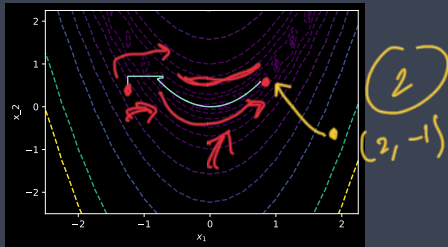
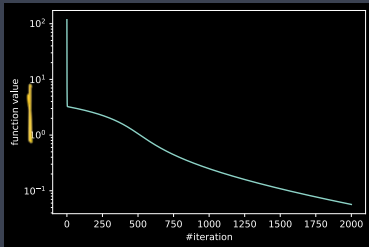
* Also observe the different steps taken too - the minimum at $x = 3$ is quite shallow so the gradient is small and steps smaller. But if choose $\alpha > 0.02$ then get oscillations around minimum at $x = 1$.

» Example: Rosenbrock Function

* $f(x) = (1.0 - x_1)^2 + 100.0 * (x_2 - x_1^2)^2$, $x = [x_1, x_2]$

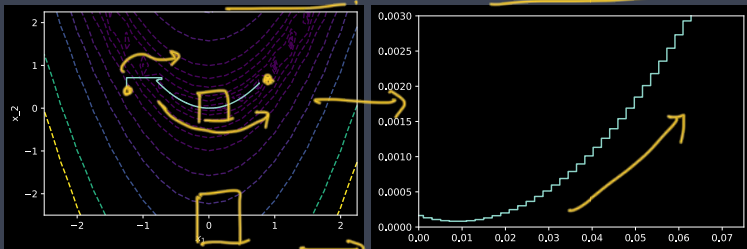


* Starting value $x_0 = [-1.25, 0.5]$, step size $\alpha = 0.001$ (note small step size, else diverges)

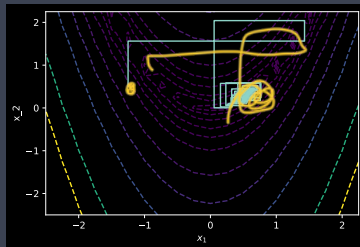


» Example: Rosenbrock Function

- * Starting value $x_0 = [-1.25, 0.5]$, step size $\alpha = 0.001$. ✓

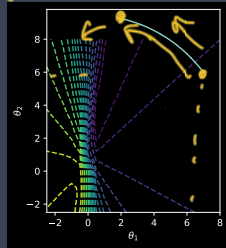
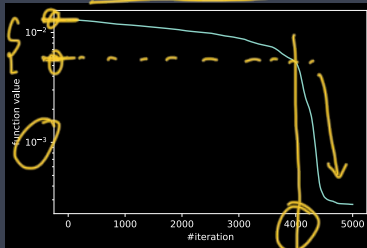


- * Need to take many small steps to move around narrow curved valley.
- * If try to take bigger steps, doesn't converge e.g. $\alpha = 0.005$:



» Example Neural Net

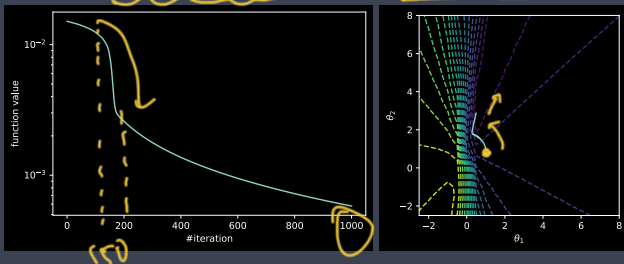
- NN
- * Toy example: $z = f(\theta_1 x + \theta_2)$, $\hat{y} = g(z)$, $\theta = [1, 5]$, f ReLu, g sigmoid, $m = 1000$ training data points with random $x^{(i)}$, quadratic loss
 - * Starting value $x_0 = [7.0, 6.0]$, step size $\alpha = 0.75$



- * Initial convergence v slow due to flat region on cost surface. Note use of larger step size to speed up convergence, and more iterations (5000)
- * Flat regions $\rightarrow$ “dead neuron” problem

» Example Neural Net

- * Toy example: $z = f(\theta_1 x + \theta_2)$, $\hat{y} = g(z)$, $\theta = [1, 5]$, f ReLu, g sigmoid, $m = 1000$ training data points with random $x^{(i)}$, quadratic loss
- * Starting value $x_0 = [1.0, 1.0]$, step size $\alpha = 0.75$



- * Different starting point $\rightarrow$ converges to different solution
- * Slower convergence once enter narrow valley with flat bottom